Vehicle 2 – Receiver / Automatic Following Code

Original code supplied by the user. Stopping distance is set to 25 cm.

```
#define BLYNK_TEMPLATE_ID   "YOUR_BLYNK_TEMPLATE_ID"
#define BLYNK_TEMPLATE_NAME "Your Project"
#define BLYNK_AUTH_TOKEN    "YOUR_BLYNK_AUTH_TOKEN"

#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <esp_now.h>
#include <esp_wifi.h>

// ---------------- WIFI ----------------
char ssid[] = "YOUR_WIFI_NAME";
char pass[] = "YOUR_WIFI_PASSWORD";

// ---------------- MOTOR DRIVER ----------------
#define ENA 25
#define IN1 26
#define IN2 27
#define ENB 13
#define IN3 12
#define IN4 14

// ---------------- HC-SR04 ----------------
#define TRIG_PIN 5
#define ECHO_PIN 18

// ---------------- WHEEL ENCODERS ----------------
#define ENCODER_LEFT  33
#define ENCODER_RIGHT 32

// Adjust this to match your encoder
const int PULSES_PER_REV = 20;

// ---------------- FOLLOWING SETTINGS ----------------
const float STOP_DISTANCE_CM = 25.0;
const float SLOW_DISTANCE_CM = 60.0;
const unsigned long FRONT_TIMEOUT_MS = 1500;

// ---------------- FRONT PACKET ----------------
// Must match the structure in the front ESP32 program.
typedef struct {
  uint8_t command;       // 0 = STOP, nonzero = moving
  uint8_t speedPercent;  // 0–100
  float rpm1;
  float rpm2;
  uint32_t sequence;
} VehiclePacket;

VehiclePacket frontData;
volatile bool newFrontData = false;
unsigned long lastFrontPacketTime = 0;

float distanceCM = 0;
float rearRPM = 0;
bool safetyStop = true;
int rearMotorPWM = 0;

volatile unsigned long leftPulses = 0;
volatile unsigned long rightPulses = 0;

unsigned long lastRPMTime = 0;
unsigned long lastBlynkUpdate = 0;

// ---------------- ENCODER INTERRUPTS ----------------
void IRAM_ATTR leftEncoderISR() {
  leftPulses++;
}

void IRAM_ATTR rightEncoderISR() {
  rightPulses++;
}

// ---------------- MOTOR CONTROL ----------------
void stopMotors() {
  digitalWrite(IN1, LOW);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, LOW);
  digitalWrite(IN4, LOW);
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
  rearMotorPWM = 0;
}

void moveForward(int pwm) {
  pwm = constrain(pwm, 0, 255);

  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  digitalWrite(IN3, HIGH);
```

```
  digitalWrite(IN4, LOW);

  analogWrite(ENA, pwm);
  analogWrite(ENB, pwm);
  rearMotorPWM = pwm;
}

// ---------------- DISTANCE MEASUREMENT ----------------
float readDistanceCM() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  unsigned long duration = pulseIn(ECHO_PIN, HIGH, 25000);

  // Missing echo is treated as unsafe
  if (duration == 0) return 0;

  return duration * 0.0343 / 2.0;
}

// ---------------- ESP-NOW RECEIVE ----------------
void onDataRecv(const esp_now_recv_info_t *info,
                const uint8_t *data,
                int len) {
  if (len != sizeof(VehiclePacket)) return;

  memcpy(&frontData, data, sizeof(frontData));
  lastFrontPacketTime = millis();
  newFrontData = true;
}

// ---------------- RPM CALCULATION ----------------
void updateRPM() {
  unsigned long now = millis();

  if (now - lastRPMTime >= 1000) {
    noInterrupts();
    unsigned long pulses = (leftPulses + rightPulses) / 2;
    leftPulses = 0;
    rightPulses = 0;
    interrupts();

    rearRPM = (pulses * 60.0) / PULSES_PER_REV;
    lastRPMTime = now;
  }
}

// ---------------- AUTOMATIC FOLLOWING ----------------
void updateFollowingControl() {
  distanceCM = readDistanceCM();

  bool communicationLost =
      (millis() - lastFrontPacketTime > FRONT_TIMEOUT_MS);

  // Stop if front vehicle brakes, communication is lost,
  // or distance measurement is invalid.
  if (communicationLost ||
      frontData.command == 0 ||
      frontData.speedPercent == 0 ||
      distanceCM <= 0) {
    safetyStop = true;
    stopMotors();
    return;
  }

  // Stop at the selected safety gap
  if (distanceCM <= STOP_DISTANCE_CM) {
    safetyStop = true;
    stopMotors();
    return;
  }

  safetyStop = false;

  int frontPWM = map(frontData.speedPercent, 0, 100, 0, 255);
  int targetPWM = frontPWM;

  // Gradually reduce rear speed when approaching
  if (distanceCM < SLOW_DISTANCE_CM) {
    float ratio =
      (distanceCM - STOP_DISTANCE_CM) /
      (SLOW_DISTANCE_CM - STOP_DISTANCE_CM);

    ratio = constrain(ratio, 0.0, 1.0);

    targetPWM = (int)(frontPWM * ratio);
  }

  moveForward(targetPWM);
}
// ---------------- BLYNK DASHBOARD ----------------
```

```
void updateBlynkDashboard() {
  if (millis() - lastBlynkUpdate < 500) return;
  lastBlynkUpdate = millis();

  // Dashboard values:
  // V5 = rear vehicle telemetry
  Blynk.virtualWrite(V5, rearRPM);
  Blynk.virtualWrite(V7, distanceCM);
  Blynk.virtualWrite(V8, safetyStop ? "STOP" : "FOLLOWING");
  Blynk.virtualWrite(V9, rearMotorPWM);
}

// ---------------- SETUP ----------------
void setup() {
  Serial.begin(115200);

  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);

  pinMode(ENCODER_LEFT, INPUT_PULLUP);
  pinMode(ENCODER_RIGHT, INPUT_PULLUP);
  attachInterrupt(digitalPinToInterrupt(ENCODER_LEFT), leftEncoderISR, RISING);
  attachInterrupt(digitalPinToInterrupt(ENCODER_RIGHT), rightEncoderISR, RISING);

  stopMotors();

  // Blynk connection
  WiFi.mode(WIFI_STA);
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

  // ESP-NOW connection
  if (esp_now_init() != ESP_OK) {
    Serial.println("ESP-NOW initialization failed");
    stopMotors();
    while (true) delay(100);
  }

  esp_now_register_recv_cb(onDataRecv);

  Serial.println("Rear vehicle ready");
}

// ---------------- MAIN LOOP ----------------
void loop() {
  Blynk.run();

  updateRPM();
  updateFollowingControl();
  updateBlynkDashboard();

  delay(20);
}
```